Vulnerability Assessment: **SQL Injection – Unsanitized user_id Parameter**
Project 4: Web Application Security Assessment
Week 3: Secure SDLC and Code Review

Vulnerability Information

- **Vulnerability Type:** SQL Injection / Unvalidated Input
- **Risk Level:** High
- **OWASP Category:** A03:2021 – Injection
- **Location:** /vulnerabilities/sqli/, line 1

Vulnerable Code Analysis

- Low

```
$query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
```

- Medium

```
$query  = "SELECT first_name, last_name FROM users WHERE user_id = $id;";

```

- High

```
$query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id' LIMIT 1;";
```

The query concatenates the variable $id directly into the SQL string without sanitization or parameterization.
There is no input validation, escaping, or type enforcement, allowing attackers to manipulate the SQL query.
This makes the code vulnerable to SQL injection, enabling unauthorized data retrieval or modification.

Variants observed include:

- Using $id without quotes → numeric-based injection possible.
- Using session variables ($_SESSION['id']) without validation → session hijacking risk.
- Lack of LIMIT or whitelisting → large-scale data extraction via UNION queries.

Secure Code Screenshot

```php
// Secure implementation using prepared statements and type casting
$id = isset($_GET['id']) ? (int) $_GET['id'] : 0;

$mysqli = new mysqli($db_host, $db_user, $db_pass, $db_name);
if ($mysqli->connect_errno) {
    error_log("DB Connection Error: " . $mysqli->connect_error);
    http_response_code(500);
    exit('Internal server error.');
}

$stmt = $mysqli->prepare("SELECT first_name, last_name FROM users WHERE user_id = ? LIMIT 1");
$stmt->bind_param("i", $id);
$stmt->execute();
$stmt->bind_result($first_name, $last_name);
$stmt->fetch();
$stmt->close();
$mysqli->close();
```

Mitigations Applied

- Replaced direct query concatenation with prepared statements.
- Added strict type casting ((int)) for numeric input.
- Implemented server-side validation for input values.
- Removed reliance on session or client-provided variables without verification.
- Limited query scope using LIMIT 1 to reduce exposure.

Recommendations for Development Team

Immediate Actions:

1. **Implement Parameterized Queries Everywhere**
    - Use prepared statements (mysqli or PDO) for all database interactions.
    - Avoid dynamic SQL string concatenation.
2. **Enforce Input Validation and Type Casting**
    - Validate all incoming data against expected formats.
    - Cast numeric inputs explicitly to integers before use.
3. **Secure Error Handling**
    - Disable detailed error messages in production.
    - Log errors securely on the server for debugging.
4. **Review All Database Access Points**
    - Conduct static code review for raw SQL usage.
    - Ensure consistent use of prepared statements across the codebase.

References

• OWASP Cheat Sheet: SQL Injection Prevention Cheat Sheet
• CWE Reference: CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
• Best Practice: Always validate, sanitize, and parameterize all user inputs before interacting with the database.